The Perceptron Trick in Code

Contents

1. Overview 2
2. The data 2
3. The code 3
4. Drawing the decision boundary 4
5. Watching the decision boundary move 5
6. The weakness: the decision boundary stops too early 6
    6.1 Comparing with logistic regression 7
    6.2 Why it matters 9
7. Summary 10
8. Sources 10
9. Key terms 10

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Perceptron trick.
- **Used here, taught in full later:** Logistic regression.
- **Compare with:** Support vector machines.

1. Overview

Key point: The perceptron trick takes about ten lines of Python. The trick finds a decision boundary (the line that separates the classes), but it stops as soon as no point is misclassified, so the boundary can end up very close to one class. **Logistic regression** (G-1120) keeps improving until the boundary sits well between them.

The previous lesson on the perceptron trick described the **perceptron trick** (G-1485): start with any line, pick random points, and move the line towards each misclassified one. The line that separates the two classes is the model's **decision boundary** (G-555): points on one side are predicted as class 1, points on the other side as class 0. This Note:

- codes the trick (section 3) and draws its decision boundary (section 4);
- watches the decision boundary move (section 5);
- compares the result with scikit-learn's `LogisticRegression` (section 6).

The comparison shows the trick's weakness, which the following Notes fix.

2. The data

Key point: 100 points with two features and two balanced classes, made with make_classification.

`make_classification` (G-1149) creates classification data, just as `make_regression` creates regression data.

Python: Creating the data.

```
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=2,
    n_informative=1, n_redundant=0, n_classes=2,
    n_clusters_per_class=1, random_state=41,
    hypercube=False, class_sep=10)
```

`X` has 100 rows and 2 columns. Each row is one **observation** (G-1374) (one record, here one point) and each column is one **feature** (G-772) (an input variable), x_1 or x_2. `y` holds the **target** (G-1949), the output we predict: the class of each observation, 50 zeros and 50 ones.

`class_sep` sets how far apart the classes are. In the figures, class 1 is green and class 0 is blue.

3. The code

> **Key point:** Add a column of 1s, start with weights of 1, and repeat 1,000 times: pick a random row, predict it with the step function, and apply w = w + lr (y − $\hat{y}$) x.

In plain words, the code does three things. First it adds a column of 1s to the data, so the intercept can be handled like any other weight. Then it starts with all weights at 1. Then it repeats one small step 1,000 times: pick a random point, predict its class, and move the weights only if the prediction was wrong.

Python: The **perceptron trick** (G-1485).

```python
def step(z):
    return 1 if z > 0 else 0

def perceptron(X, y, lr=0.1, loops=1000, seed=0):
    rng = np.random.default_rng(seed)
    X = np.insert(X, 0, 1, axis=1)      # column of 1s
    weights = np.ones(X.shape[1])       # [w0, w1, w2]
    for _ in range(loops):
        j = rng.integers(0, X.shape[0])             # random row
        y_hat = step(X[j] @ weights)                # predict it
        weights = weights + lr * (y[j] - y_hat) * X[j]
    return weights[0], weights[1:]

intercept_, coef_ = perceptron(X, y)
```

Line by line:

- `np.insert(X, 0, 1, axis=1)` puts a 1 at the start of every row, so each row becomes $(1, x_1, x_2)$. The first weight then acts as the **intercept** (G-960), also called the **bias** of the perceptron (G-284).
- `weights` starts as $(1, 1, 1)$: the **decision boundary** (G-555) $1 + x_1 + x_2 = 0$.
- In each loop, `X[j] @ weights` is the dot product (multiply matching entries and add) $w_0 + w_1 x_1 + w_2 x_2$ for row j, and `step` turns it into a prediction of 1 or 0. This 0-or-1 cut at zero is the **step function** (G-1889).
- The update changes the weights only when $y \neq \hat{y}$ (see one update rule).

With seed 0, the result is $w_0 = 1.00$, $w_1 = 1.34$, $w_2 = 0.19$, and no training point is misclassified.

Figure 1 tracks the three weights through all 1,000 loops. Watch them move only in 6 jumps and then stay flat from loop 108 to the end.

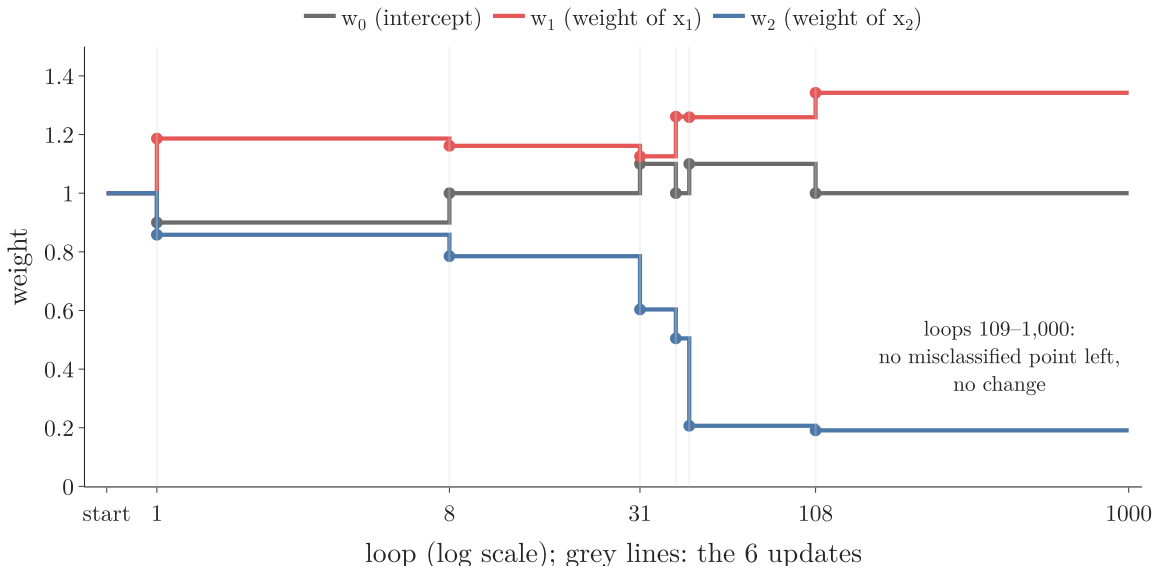


Figure 1: The weights w0, w1 and w2 after every loop of the seed-0 run (log scale for the loop number). They change only at loops 1, 8, 31, 40, 44 and 108, when the picked point was misclassified; loops 109 to 1,000 change nothing.

> **Extra:** The fixed **random seed** (G-1617) makes the random picks the same on every run, so the results repeat. Without it, every run gives a slightly different decision boundary.

4. Drawing the decision boundary

> **Key point:** To plot $w_0 + w_1x_1 + w_2x_2 = 0$, solve for x_2: slope $m = -w_1/w_2$, **intercept** (G-960) $b = -w_0/w_2$.

With two features the **decision boundary** (G-555) is a straight line. To draw it as $x_2 = mx_1 + b$, solve $w_0 + w_1x_1 + w_2x_2 = 0$ for x_2:

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

With the weights above, $w_0 = 1.00$, $w_1 = 1.34$ and $w_2 = 0.19$ (rounded; the unrounded values give the numbers below), one line per value:

$$m = -\frac{w_1}{w_2} = -\frac{1.34}{0.19} \approx -7.02$$

$$b = -\frac{w_0}{w_2} = -\frac{1.00}{0.19} \approx -5.23$$

The boundary is steep because w_2 is small: the classes are separated mainly by x_1.

Figure 2 draws this decision boundary over the data. Watch where it crosses $x_1 = 0$, at $b = -5.23$, and how steeply it climbs between the two classes.

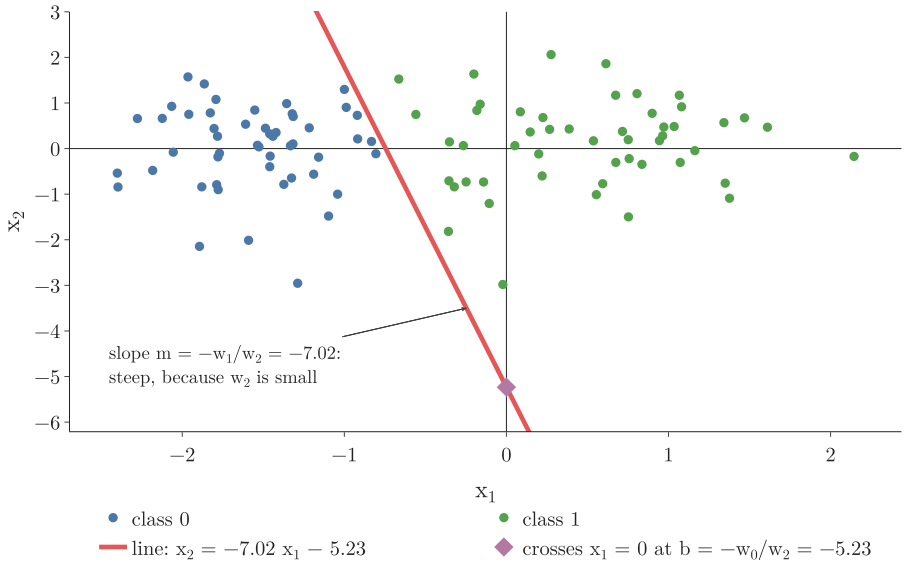


Figure 2: The final weights (1.00, 1.34, 0.19) drawn as the line x2 = −7.02 x1 − 5.23. The diamond marks the intercept b = −w0/w2; the slope −w1/w2 is steep because w2 is small.

5. Watching the decision boundary move

> **Key point:** The **decision boundary** (G-555) moved only 6 times in 1,000 loops. After loop 108 every point was on its correct side, so nothing changed again.

Figure 3 animates the run. A red ring marks each picked point that was misclassified, and the decision boundary jumps towards it.

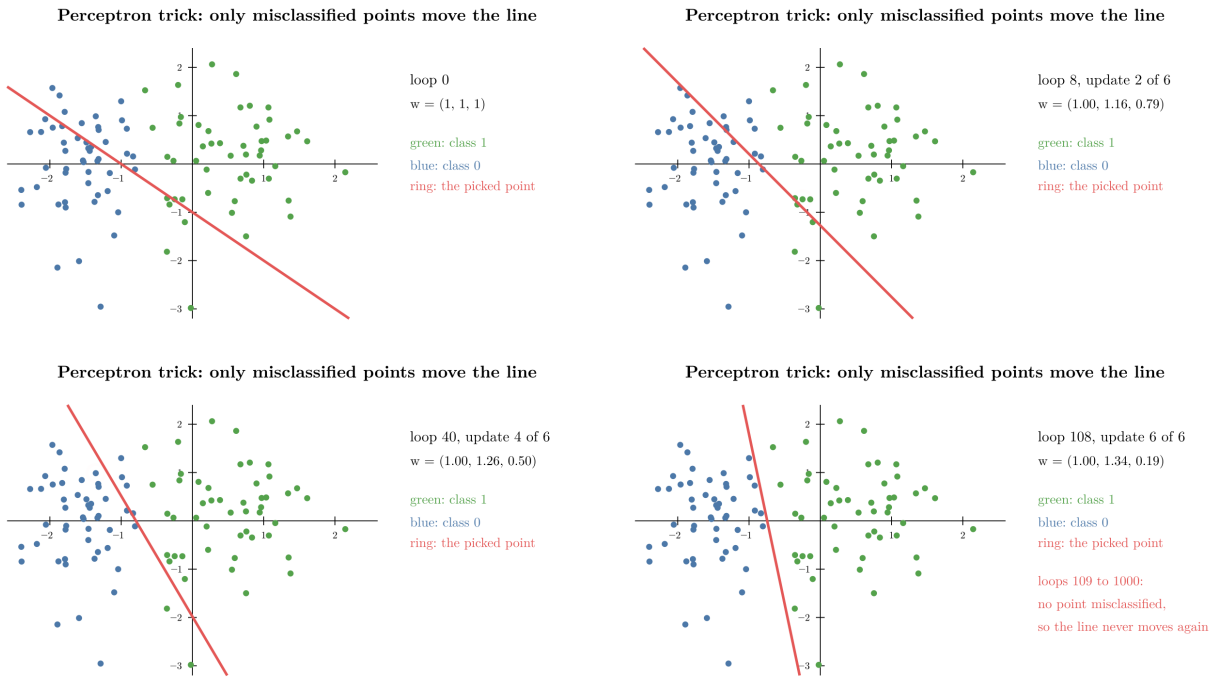


Figure 3: The perceptron trick on 100 points: the decision boundary after 0, 2, 4 and 6 updates

Update	Loop	Picked point's class	Weights (w_0, w_1, w_2) after
start	0		(1, 1, 1)
1	1	0	(0.90, 1.19, 0.86)
2	8	1	(1.00, 1.16, 0.79)
3	31	1	(1.10, 1.13, 0.60)
4	40	0	(1.00, 1.26, 0.50)
5	44	1	(1.10, 1.26, 0.21)
6	108	0	(1.00, 1.34, 0.19)

Most loops pick a point that is already correctly classified: then $y - \hat{y} = 0$, the update adds nothing, and the boundary stays still. Only a misclassified point gives $y - \hat{y} = \pm 1$ and moves it. After update 6 there are no misclassified points left, so loops 109 to 1,000 change nothing.

6. The weakness: the decision boundary stops too early

> **Key point:** Five random orders give five different decision boundaries, some almost touching one class. **Logistic regression** (G-1120) places its boundary with a fair margin to both classes.

6.1 Comparing with logistic regression

> **Key point:** Fit scikit-learn's LogisticRegression on the same data and draw both decision boundaries.

We train scikit-learn's `LogisticRegression` on the same 100 points and turn its weights into a line with the formulas of section 4.

> **Python:** Logistic regression in scikit-learn.

```python
from sklearn.linear_model import LogisticRegression

lor = LogisticRegression(C=100)
lor.fit(X, y)
m = -lor.coef_[0][0] / lor.coef_[0][1]
b = -lor.intercept_[0] / lor.coef_[0][1]
```

Figure 4 draws the perceptron's decision boundaries from five random orders (seeds 0 to 4) in red and the **logistic regression** (G-1120) boundary in black.

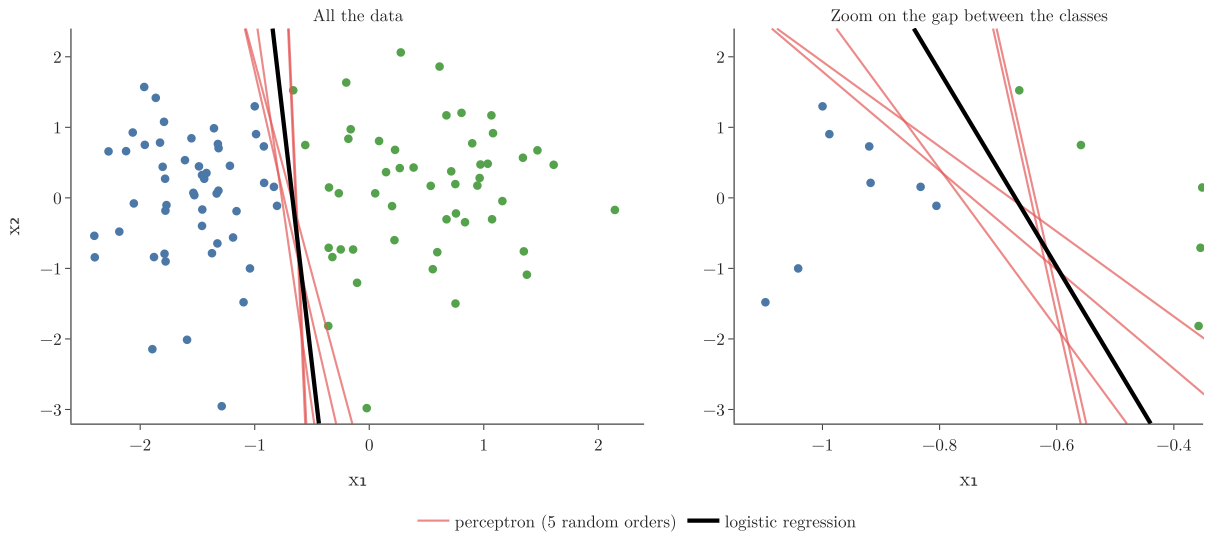


Figure 4: Five perceptron decision boundaries and the logistic regression boundary, with a zoom on the gap

All six boundaries separate the training data perfectly. The difference is where they sit in the gap between the classes. The distance from a boundary to the nearest point of a class is its **margin** (G-1159) on that side. The table gives both margins:

Model	Gap to class 1	Gap to class 0
Perceptron, seed 0	0.127	0.065
Perceptron, seed 1	0.234	0.052

Model	Gap to class 1	Gap to class 0
Perceptron, seed 2	0.021	0.164
Perceptron, seed 3	0.020	0.104
Perceptron, seed 4	0.015	0.171
Logistic regression	0.117	0.143

Three of the five perceptron boundaries pass within about 0.02 of a green point. The logistic regression boundary keeps a similar margin to both classes.

Figure 5 shows why the two end up in different places. Both models start from the same weights $(1, 1, 1)$. On the left is the perceptron run of section 5. On the right, logistic regression is trained step by step with gradient descent (the method of the logistic gradient descent). Watch the right panel after the left one has stopped.

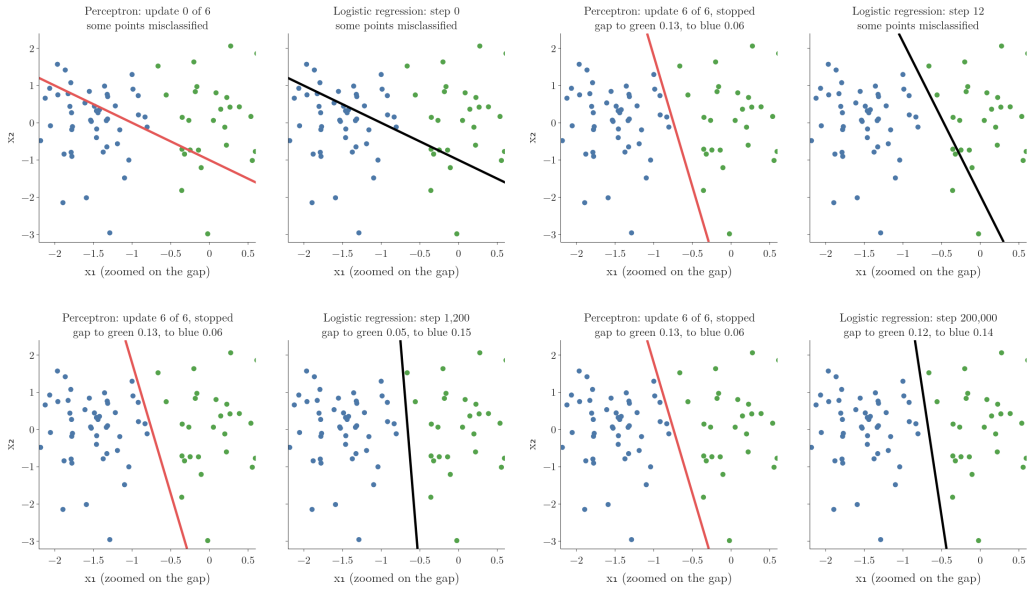


Figure 5: The perceptron trick (left, seed 0) and logistic regression trained by gradient descent (right) on the same 100 points, zoomed on the gap between the classes. The perceptron's decision boundary stops after its 6th update, 0.06 from the nearest blue point. The logistic regression boundary keeps moving after every point is already on its correct side, and ends near the middle of the gap (0.12 and 0.14)

1. **Perceptron:** after 6 updates no point is misclassified, so nothing can move the boundary again. The boundary stays where the last update left it, 0.13 from the nearest green point and 0.06 from the nearest blue point.
2. **Logistic regression:** the boundary also reaches zero misclassified points early, but it does not stop there. Every later step still moves it a little, until it settles at 0.12 and 0.14: the same boundary scikit-learn finds.

6.2 Why it matters

> **Key point:** A **decision boundary** (G-555) that hugs one class misclassifies new points from that class easily. Zero training error is not the same as a good model.

The **perceptron trick** (G-1485) stops improving as soon as the training error is zero: its only goal is "no misclassified points". Where exactly the boundary ends up depends on the random order of picks (Bishop §4.1.7).

A new class-1 point that falls just next to the green cluster could easily land on the wrong side of a boundary that hugs it. A boundary in the middle of the gap leaves room for such points. Think of parking a car in a garage: parked tight against one wall, the smallest drift scrapes the paint; parked in the middle, there is room on both sides.

A test on real data confirms this. On the iris flowers, setosa against versicolor (two species a straight line can separate), the perceptron's boundaries misclassified 5.5% of new flowers on average, against 1.2% for **logistic regression** (G-1120). How well a model does on new data is its **generalisation** (G-838). The same idea drives the maximal margin classifier, taught in the **support vector machine** (G-1921): a boundary with a wide **margin** (G-1159) on the training data should also keep a wide margin on new data, and so classify it correctly (ISL §9.1.3).

> **Extra:** The test, in the notebook: sepal length and sepal width (standardised), 10 random training flowers per species, the other 80 flowers as the test set, averaged over 100 random draws.

Logistic regression keeps adjusting the boundary even when every training point is already correct, until the boundary is placed as well as possible. How it decides what "as well as possible" means is the subject of the next Notes: the sigmoid function, and then the loss function.

> **Extra:** `C=100` makes scikit-learn's regularisation weak: `C` is the inverse of the penalty strength, and the default penalty is a Ridge-like L2 penalty (scikit-learn docs, `LogisticRegression`). With the default `C=1` the penalty keeps the weights smaller, which on this data moves the boundary slightly: it then misclassifies one training point by a hair (0.009). The hyperparameters of logistic regression are covered in the strength C.

7. Summary

- The **perceptron trick** (G-1485) in code: a column of 1s, weights of 1, and 1,000 loops of $w \leftarrow w + \eta(y - \hat{y})x$; the column of 1s lets the **intercept** (G-960) be learned like any other weight.
- The **decision boundary** (G-555) is drawn with slope $-w_1/w_2$ and intercept $-w_0/w_2$, because solving $w_0 + w_1x_1 + w_2x_2 = 0$ for x_2 gives a line you can plot.
- On the example, the boundary moved only 6 times; after that, no point was misclassified and it stopped, because a correct point gives $y - \hat{y} = 0$ and adds nothing.
- Its final position depends on the random order and can hug one class, so a new point from that class can easily land on the wrong side.
- **Logistic regression** (G-1120) places the boundary with a fair margin to both classes, which generalises better (iris test: 1.2% errors against 5.5% for the perceptron), because it keeps adjusting after the training error reaches zero.
- So the code works, but zero training error is not a good model; the next Notes give logistic regression the sigmoid and the loss that place the boundary well.

8. Sources

Built from

- CampusX, "Logistic Regression Part 2 | Perceptron Trick Code", YouTube, https://www.youtube.com/watch?v=tLezwPKvPK4

Other references

- **scikit-learn docs:** `sklearn.linear_model.LogisticRegression` (parameters C and penalty), scikit-learn 1.9.
- **Bishop:** Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006. Section 4.1.7, p. 194 (the perceptron's solution depends on the starting weights and the order of the points).
- **ISL:** James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021. Section 9.1.3, p. 371.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
make_classification (G-1149)	scikit-learn function that creates random classification data.
Margin (gap) (G-1159)	The distance from a separating line to the nearest point of a class.
Generalisation (generalise) (G-838)	How well a model performs on new data it was not trained on.

Term	Meaning
class_sep (G-390)	make_classification setting for how far apart the classes are.
Logistic regression (G-1120)	A classification algorithm that passes a weighted sum of the inputs through the sigmoid to give the probability of a class; the line (or plane) where that probability is 0.5 is the boundary it learns between the classes.
Perceptron trick (G-1485)	Moving a line towards each misclassified point until the classes are separated.
Decision boundary (G-555)	A line or curve that separates the classes in classification.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Intercept (G-960)	The line's value when the input is 0; b in $y = mx + b$.
Bias (of a perceptron) (G-284)	The weight on a constant input of 1; it shifts the boundary away from the origin.
Step function (G-1889)	A function that outputs 1 for positive inputs and 0 otherwise; the classic perceptron uses it to turn its score into a class.
Random seed (G-1617)	A number that fixes a random number generator so that a run can be repeated exactly.
Support vector machine (SVM) (G-1921)	A classifier that separates the classes with the line, or flat surface in more dimensions (a hyperplane), that leaves the widest gap (margin) between the classes.